

Sección 4 — Juegos

Métricas: el grafo de Hamming 1 del corpus, la tasa de éxito del backtracking y el determinismo por semilla, medidos sobre los motores reales

1. Qué se mide y cómo

Todas las cifras de este documento salen de **ejecutar los motores de producción** (src/engine/escalera.js y crucigrama.js) con `npm run simular-juegos` (simulacion/juegos-stats.mjs → docs/datos-juegos.json, mismo patrón que la comparación SM-2/Leitner de la Sección 2). No hay reimplementación paralela: si el motor cambia, las métricas cambian con él. Para la **escalera** se caracteriza el grafo de palabras del corpus (¿existen retos? ¿de qué dificultades?); para el **crucigrama**, un barrido de 200 semillas por tamaño mide tasa de éxito, densidad y coste.

2. Escalera: el grafo de Hamming 1 del corpus

2.1 Formalización

$G_L = (V, E)$ con V = palabras de longitud L del léxico (solo ASCII) y $\{u, v\} \in E \Leftrightarrow d_H(u, v) = 1$. Un reto de k pasos es un par con $\text{dist}_G(\text{origen}, \text{destino}) = k$; BFS certifica la distancia en $O(V + E)$. La construcción del grafo usa cubetas comodín (L máscaras por palabra): $O(n \cdot L)$ contra el $O(n^2 \cdot L)$ de comparar parejas.

2.2 El grafo medido, por longitud

L	Nodos	Aristas	Componentes	Gigante	Aisladas	Diámetro*	Dist. media*
3	97	136	11	62	5	19	6.08
4	268	277	95	115	75	16	6.88
5	445	315	223	64	176	12	4.46

* Diámetro y distancia media sobre pares alcanzables (dentro de cada componente). El grafo NO es conexo — el léxico de un corpus literario no es un diccionario exhaustivo — pero la **componente gigante** basta: el generador de retos solo elige pares a distancia exacta k , que por definición viven en la misma componente. Nótese la forma de campana con la longitud: con $L=3$ hay pocas palabras pero muy conectadas; con $L=5$ hay más palabras pero el espacio 26^5 está mucho más vacío y las vecinas escasean.

2.3 ¿Existen retos para cada dificultad?

L	Pares alcanzables	a distancia 3	a distancia 4	a distancia 5	a distancia 6
3	2033	237	234	194	180
4	6845	533	624	656	672
5	2832	413	415	397	329

Los cuatro niveles de dificultad que ofrece la UI (3–6 pasos) tienen cientos o miles de pares candidatos en cada longitud: el selector nunca cae en vacío. La cola de la distribución (hasta el diámetro, 12–19 según L) queda como margen para dificultades futuras.

3. Crucigrama: el backtracking medido

3.1 Formalización

Colocar n palabras entrelazadas es un problema de satisfacción de restricciones (variables = palabras; dominios = colocaciones legales; restricciones = coincidencia de letra en cruces, extremos libres, sin contactos laterales, ≥ 1 cruce por palabra). La búsqueda es en profundidad con retroceso: el anclaje en cruces reduce el dominio de cada variable de $O(\text{filas} \cdot \text{columnas} \cdot 2)$ a las casillas que ya contienen una letra de la palabra, y el orden «más cruces primero» prioriza tableros densos. Presupuesto de 5.000 nodos y degradación $n \rightarrow n-1$ garantizan terminación con salida.

3.2 Barrido de semillas

n pedido	Semillas	Tasa de éxito	Palabras medias	Cruces medios	Densidad media	ms / tablero
6	200	100.0%	6	5.08	0.285	0.32
8	200	100.0%	8	7.15	0.268	0.25
10	200	100.0%	10	9.19	0.255	0.38

Con el pool real (400 entradas, 30 candidatas barajadas por semilla) el backtracking coloca las n palabras pedidas en el **100% de las semillas** probadas y en décimas de milisegundo: el presupuesto de nodos es un seguro, no un coste. Cruces medios $\approx n - 1$ con excedente pequeño: el tablero típico es un árbol de palabras con algún cruce extra — exactamente lo que las reglas permiten sin forzar rejillas imposibles. La densidad (casillas ocupadas / rectángulo envolvente) baja suavemente con n : tableros más grandes se dispersan, pero no colapsan.

4. Codenames: la semilla como canal

La partida entera se codifica en 6 caracteres: 2 de vocabulario (idioma $\times$ nivel) y 4 de semilla en alfabeto de 36 símbolos $\rightarrow 36^4 = 1.679.616$ tableros por vocabulario. El LCG ($a = 1103515245$, $c = 12345$, $m = 2^{31}$, los parámetros de la libc clásica) genera la permutación de palabras (Fisher–Yates) y el reparto de colores 9/8/7/1. La propiedad que importa no es la calidad criptográfica (irrelevante aquí) sino el **determinismo multiplataforma**: la aritmética entera de JS reproduce el mismo tablero en cualquier dispositivo, y la semilla sustituye al servidor.

```
x_{i+1} = (a \cdot x_i + c) \bmod m \quad \text{con } x_0 = \text{hash(semilla)}
36^4 = 1\,679\,616 \text{ tableros/vocabulario} \cdot 13 \text{ vocabularios}
```

5. Determinismo transversal

Los tres juegos comparten el mismo generador (board.js) y la misma disciplina que las Secciones 2 y 3: **el azar solo entra por la semilla**. Consecuencias medibles: los tests de los motores pueden afirmar igualdad exacta de retos y tableros (toEqual entre dos llamadas con la misma semilla); un reto de escalera o un crucigrama son compartibles por URL/string sin estado en servidor; y las estadísticas de este documento son reproducibles con `npm run simular-juegos`.

6. Limitaciones y trabajo futuro

El grafo hereda el corpus. Las glosas y las palabras vienen de lecturas literarias: hay formas conjugadas raras en la escalera y alguna glosa contextual («hoch» $\rightarrow$ «anticiclón» si en el texto era el sustantivo Hoch). El filtro es léxico, no semántico. **Sin umlauts.** Excluir ä/ö/ü/ß simplifica el tecleo pero recorta el alemán real; una alternativa futura es aceptar ae/oe/ue como equivalentes. **Crucigrama sin rejilla simétrica.** Se genera estilo «entrelazado libre», no la rejilla simétrica de periódico (eso exigiría diccionarios mucho mayores y otro algoritmo, p. ej. dancing links). **Futuro:** retos diarios (semilla = fecha), palabras de la bolsa del usuario como pool del crucigrama (conectaría con la Sección 2), y dificultad de la escalera por rareza de las palabras además de por distancia.